

MeetFlow

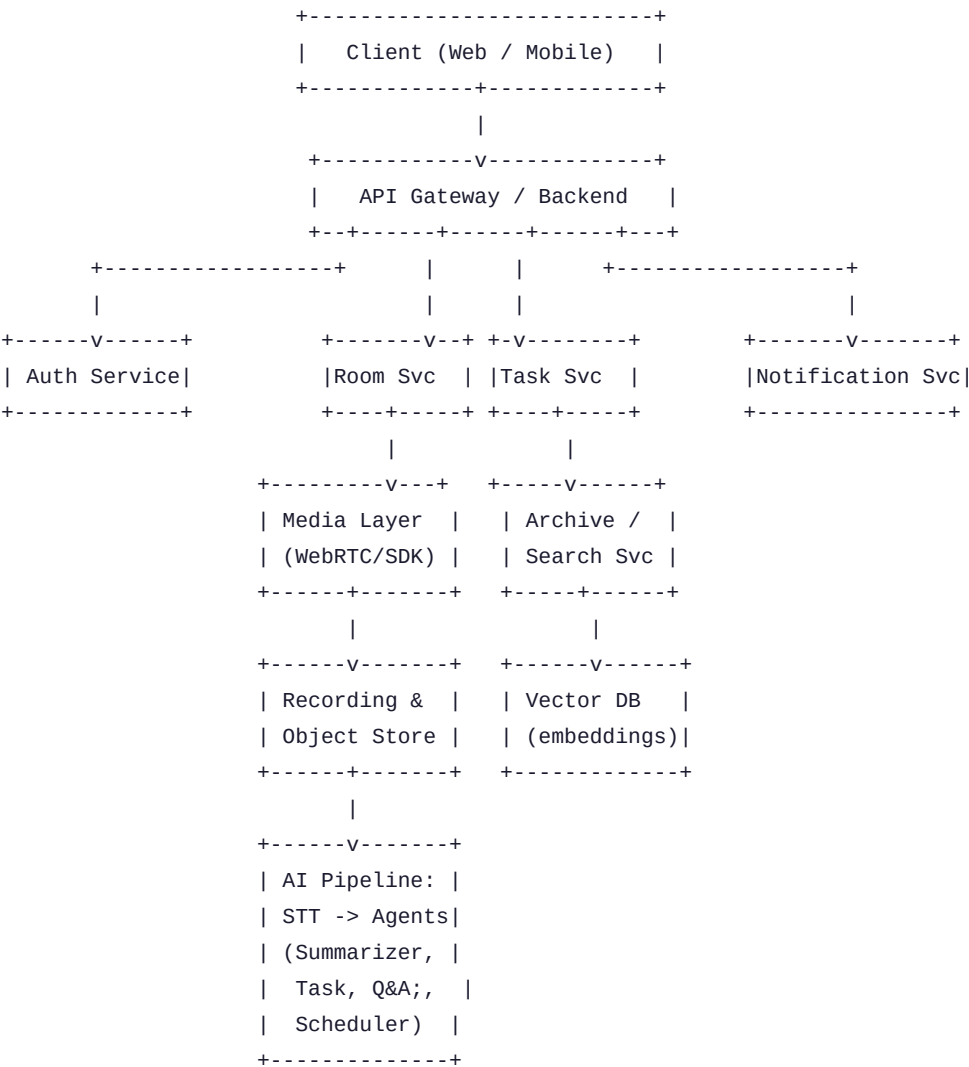
Backend Schema & System Architecture Document

Version 1.0 | AI-Powered Meeting Platform for Colleges

1. Document Purpose

This document defines MeetFlow's system architecture and backend data schema: the services that make up the backend, how they interact, and the database tables (with fields and relationships) needed to support every feature in the PRD/TRD. It is intended as a direct reference for implementation.

2. System Architecture



Relational DB (PostgreSQL) is used by Auth, Room, Task, Notification, and Archive services.

2.1 Service Responsibilities

- Auth Service — Google OAuth/email sign-up, session tokens, stores age-range and language preference.
- Room Service — room/meeting lifecycle: create, schedule, reschedule (preserving room_code), cancel, settings, join-mode enforcement, waiting room.
- Media Layer — real-time audio/video/screen-share via WebRTC or a managed SDK; active-speaker + pin state.
- Recording & Object Store — stores raw recordings, chat logs, and generated highlight reels.
- AI Pipeline — speech-to-text, then Summarizer / Task / Q&A; / Scheduler agents operating on the transcript.
- Task Service — task creation (manual/select-all/auto-detected), progress tracking, overdue reminders.
- Notification Service — calendar-scoped reminders, task-overdue alerts, waiting-room/admit notices.
- Archive/Search Service — attended-meeting access (no code), one-time absentee unlock via room_code, topic search.

3. Database Schema (Relational)

3.1 users

Column	Type	Notes
user_id	UUID (PK)	Primary key
email	string	Unique; used for login
auth_provider	enum	google / email
age_range	enum	below_18 / above_18
language_pref	string	e.g. en, fr
display_name	string	Account-level default display name
created_at	timestamp	

3.2 rooms (meeting series / template)

Column	Type	Notes
room_id	UUID (PK)	Primary key
room_code	string	Unique, shareable code; stable across reschedules
host_id	UUID (FK -> users)	Owner/host of this room
title	string	Meeting/topic title
password_hash	string, nullable	Set only if Meeting Password is enabled
waiting_room_enabled	boolean	
join_before_host	boolean	
auto_record	boolean	
join_mode	enum	auto_admit / manual_approval
is_recurring	boolean	

class_or_group_label	string, nullable	For per-class/group dashboard grouping
created_at	timestamp	

3.3 meeting_instances (a specific occurrence of a room)

Column	Type	Notes
meeting_instance_id	UUID (PK)	Primary key
room_id	UUID (FK -> rooms)	Parent room
scheduled_start	timestamp	Updated on reschedule; room_code unchanged
scheduled_end	timestamp, nullable	
status	enum	scheduled / live / ended / cancelled
recording_url	string, nullable	Full recording, once available
highlight_reel_url	string, nullable	AI-generated short clip
transcript_ref	string, nullable	Pointer to transcript store

3.4 participants (attendance record)

Column	Type	Notes
participant_id	UUID (PK)	Primary key
meeting_instance_id	UUID (FK)	
user_id	UUID (FK -> users)	
role_in_meeting	enum	host / co_host / participant
joined_at / left_at	timestamp, nullable	Attendance timestamps
display_name_override	string, nullable	In-meeting name change
profile_photo_override	string, nullable	In-meeting photo change

3.5 calendar_entries (per-user calendar)

Column	Type	Notes
calendar_entry_id	UUID (PK)	Primary key
user_id	UUID (FK -> users)	Owner of this calendar entry
meeting_instance_id	UUID (FK)	
added_via	enum	host_default / room_code_added
reminder_enabled	boolean	Drives Reminder Agent eligibility

3.6 archive_access_grants (absentee one-time unlock)

Column	Type	Notes
grant_id	UUID (PK)	Primary key
user_id	UUID (FK -> users)	Absentee who unlocked access
meeting_instance_id	UUID (FK)	
unlocked_at	timestamp	First time room_code was verified

3.7 summaries

Column	Type	Notes
summary_id	UUID (PK)	Primary key
meeting_instance_id	UUID (FK)	
ai_draft_text	text	Original Summarizer Agent output
current_text	text	Host-edited/approved text
status	enum	pending_review / approved
visibility	enum	absentees_only / everyone
published_at	timestamp, nullable	

3.8 chat_messages

Column	Type	Notes
message_id	UUID (PK)	Primary key
meeting_instance_id	UUID (FK)	
sender_user_id	UUID (FK -> users)	
content_type	enum	text / poll / photo / document / contact
content	text / JSON	Message body or poll definition + votes
sent_at	timestamp	

3.9 tasks

Column	Type	Notes
task_id	UUID (PK)	Primary key
meeting_instance_id	UUID (FK), nullable	Null if not tied to a specific meeting
assigned_by_user_id	UUID (FK -> users)	Host who assigned it (null if auto-detected)
assignee_user_id	UUID (FK -> users)	
source	enum	manual / select_all / auto_detected
title	string	

due_at	timestamp	Date AND time
progress_percent	integer	0-100
status	enum	pending / in_progress / done / overdue
attachment_url	string, nullable	

3.10 qa_questions

Column	Type	Notes
question_id	UUID (PK)	Primary key
asked_by_user_id	UUID (FK -> users)	
meeting_instance_id	UUID (FK), nullable	Null for multi-meeting search
question_text	text	
ai_answer_text	text, nullable	
confidence_score	float, nullable	
status	enum	answered / escalated
escalated_to_host_id	UUID (FK -> users), nullable	Chosen by the asker when escalated
host_answer_text	text, nullable	Saved for future auto-answers
asked_at	timestamp	

3.11 faculty_availability

Column	Type	Notes
availability_id	UUID (PK)	Primary key
user_id	UUID (FK -> users)	
status	enum	available / busy / do_not_disturb
updated_at	timestamp	

3.12 notifications

Column	Type	Notes
notification_id	UUID (PK)	Primary key
user_id	UUID (FK -> users)	Recipient
type	enum	reminder / task_overdue / waiting_room / escalation / summary_published
reference_id	UUID	Points to the relevant meeting_instance_id / task_id / question_id
sent_at	timestamp	
read_at	timestamp, nullable	

4. Key Entity Relationships

- users 1—N rooms (a user can host many rooms)
- rooms 1—N meeting_instances (a recurring room has many dated occurrences; room_code stays constant)
- meeting_instances 1—N participants (attendance records link users to a specific occurrence)
- users 1—N calendar_entries — N—1 meeting_instances (a user's personal calendar, decoupled from mere invitation)
- meeting_instances 1—1 summaries (each occurrence has at most one summary record, versioned via ai_draft_text vs. current_text)
- meeting_instances 1—N chat_messages, 1—N tasks, 1—N qa_questions
- users 1—N archive_access_grants — N—1 meeting_instances (absentee unlock records)

5. Access Control Rules (Enforced at Query Level)

- A host's queries for meeting_instances, chat_messages, tasks, and qa_questions are always scoped to room_id where rooms.host_id = current_user_id — never another host's rooms.
- A participant/absentee sees a meeting_instance in their Archive if EITHER: a participants row exists for (user_id, meeting_instance_id) [attended], OR an archive_access_grants row exists for (user_id, meeting_instance_id) [absentee, previously unlocked].
- summaries.status = pending_review rows are only ever returned to the host_id of the owning room — never to participants, until status = approved.
- qa_questions.host_answer_text, once saved, is reused to auto-answer future matching questions without needing host_id selection again for that exact question pattern.

6. Reschedule Data Flow (Illustrative)

This flow shows how a reschedule propagates without changing the room_code, per the product requirement:

1. Host calls PATCH /rooms/{room_id}/reschedule with new_start_time
2. Room Service updates meeting_instances.scheduled_start (room_code UNCHANGED)
3. Room Service queries calendar_entries WHERE meeting_instance_id = X
4. For each matching user_id -> Notification Service pushes a 'meeting_rescheduled' update
5. Client calendar view re-fetches and reflects the new date/time automatically